

Clab-1 Report

ENGN4528

Name: Yiran Wang

UID: U7079256

24/03/2022

Task-1: Python Warm-up. (2 marks):

Describe (in words where appropriate) the result/function of each of the following commands of your preferred language in the report. Please utilize the inbuilt help() command if you are unfamiliar with these functions.

(1) `a = np.array([[1, 2, 3], [5, 2, 20]])`

It assigns a 2D array to a variable called "a", and the array consists of two rows and three columns. The first row is [1,2,3] and the second row is [5,2,20]
The array is of size (2,3)

(2) `b = a[1, :]`

It assigns the second row of 2d array 'a' to the variable 'b'. 'b' now is a 1d array [5,2,20]
b of size (3,)

(3) `f = np.random.randn(200,1)`

the built-in function from the NumPy library is used at this stage. Python will create a series of numeric numbers according to the given parameters, and the output satisfies the standard normal distribution. At this stage, a 2d array of shape (200,1)[a column array of length 200] which is filled with random numbers satisfied the standard normal distribution. This array is assigned to variable 'f'
f of size (200,1)

(4) `g = f[f > 0]`

all elements in the array 'f' which are bigger than 0 are filtered out and reconstruction into a 1d array(as a row array). The output array is assigned to variable 'g'. g of size (x,) where x is the number of elements bigger than 0.

(5) `x = np.zeros(50) + 0.5`

This is also a built-in function from the NumPy library. An array filled with 0 is created at the beginning(a row array length of 50). After that, every element in this array is added with 0.5. Then we get an array filled with 0.5 and the array is assigned to variable 'x'
X of size (50,)

(6) `y = 0.5 * np.ones([1, len(x)])`

- 1) `len()` is a built-in function that returns the length of the input array. And `len(x)` will return an integer '50'
- 2) `np.ones()` is a built-in function from library numpy. This function will create an array consists of 1, and the dimension of the array is controlled by the input parameters. `[1, len(x)] -> [1, 50]`
- 3) after the array is created, all elements are multiplied by 0.5, and we get an array of size (1,50) consisting of 0.5
- 4) assigns the array to variable 'y'
- 5) y of size (1,50)

(7) `z = x + y`

Although y is a kind of '2d' array, it only consists of one row, then it is possible

to just add with x, a '1d' array.

The two arrays just add up according to the normal vector adding operation.

The result of $x+y$ is an array that only consists of '1'. Finally, it is assigned to the variable 'z'

Note that, the z is also a kind of '2d' array, but we could be regarded it as a 1d array as it only has one row.

z of size (1,50)

(8) `a = np.linspace(1,200)`

create an arithmetic sequence. The default length is 50, and the input parameters show that the sequence starts at 1 and end at 200

"a" is of size (50,)

(9) `b = a[::-1]`

reshape the array 'a'. This operation transforms a to its 'inverse'. The order of the elements flips around. At this stage, the result array is an arithmetic sequence that starts from 200 ends at 1 with a length of 50.

Array 'a' is an increasing sequence, and array 'b' is a decreasing sequence

B of size (50,)

(10) `b[b < 35] = 0`

Modify the elements in the array 'b'. We can find that all elements that are smaller than 35 are assigned to 0 after this operation. On the other hand, because "b" refers to "a" and shares the same reference. Array 'a' is also modified in the same way as array 'b'

Task-2: Basic Image I/O (2 marks)

In this task, you are asked to:

- Given the three images shown in Figure 1:
 1. Save them to a size of 1024 columns x 720 rows and save them to JPG image files named 'image1.jpg', 'image2.jpg' and 'image3.jpg'. (0 marks).
 2. Using image1.jpg, develop short computer code that does the following tasks:
 - a. Read this image from its JPG file and resize the image to 384 x 256 in columns x rows (0.2 marks).
 - b. Convert the color image into three grayscale channels, i.e., R, G, B images, and display each of the three grayscale images separately (0.2 marks for each channel, 0.6 marks in total).
 - c. Compute the histograms for each of the grayscale images and display the 3 histograms (0.2 marks for each histogram, 0.6 marks in total).
 - d. Apply histogram equalization to the resized image and its three grayscale channels, and then display the 4 histograms equalization image (0.15 marks for each histogram, 0.6 marks in total). (Hint: you can use inbuilt functions for implementing histogram equalization. e.g., `histeq()` in Matlab or

`cv2.equalizeHist()` in Python).

1.

- Use `cv2.imread` to read the image according to its relative paths “images/image1.jpg”
- Use `cv2.resize(image_name, (xsize, ysize))` the first parameter is for the name of the original image which will be resized by cv2. The second parameter is for the output size of the target image. `xsize` is for the length of the image and `ysize` is for the height of the image.
- Use `cv2.imwrite(output path, output image)` to store the images. The first is for the path where cv2 will output the resulting image, we use the same relative path because we want to overwrite the original images. The second parameter is the image object which will be stored.

2.

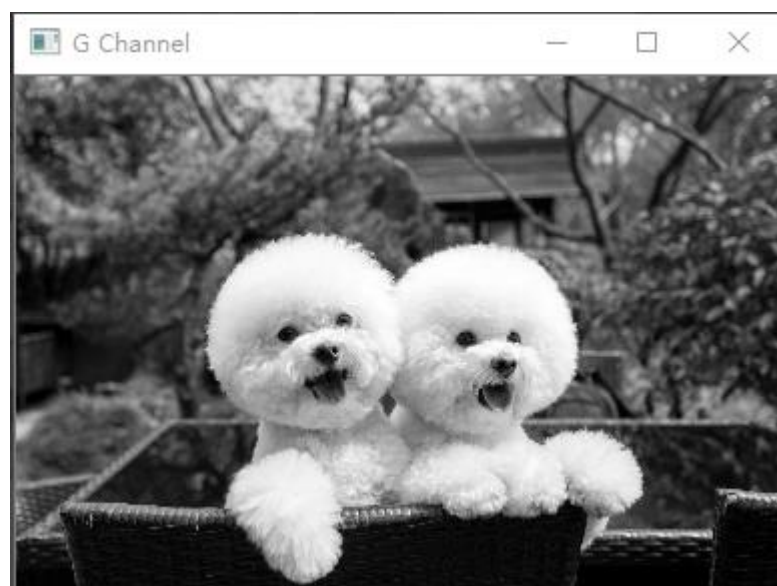
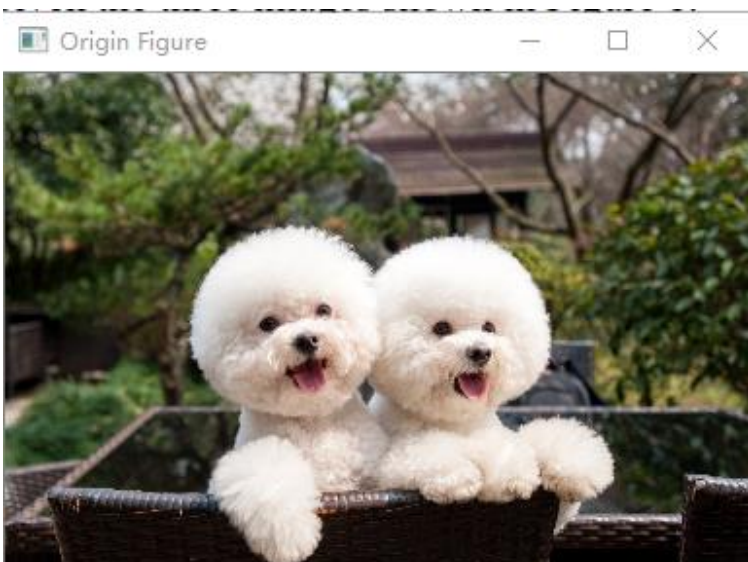
a.

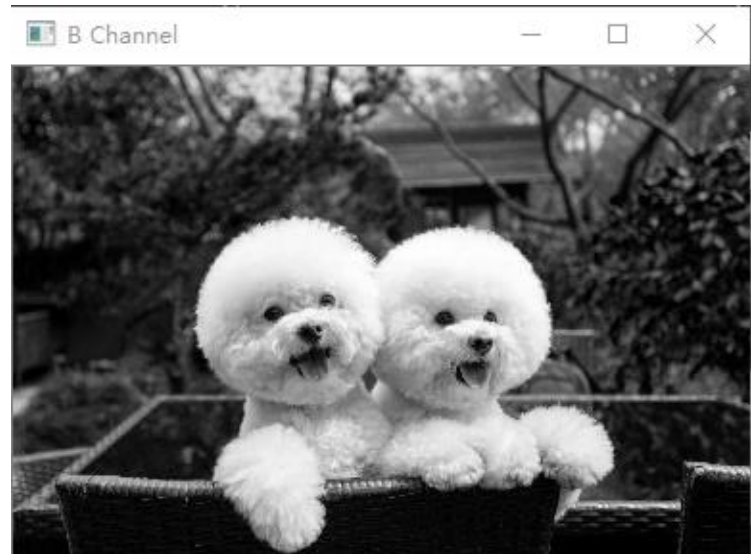
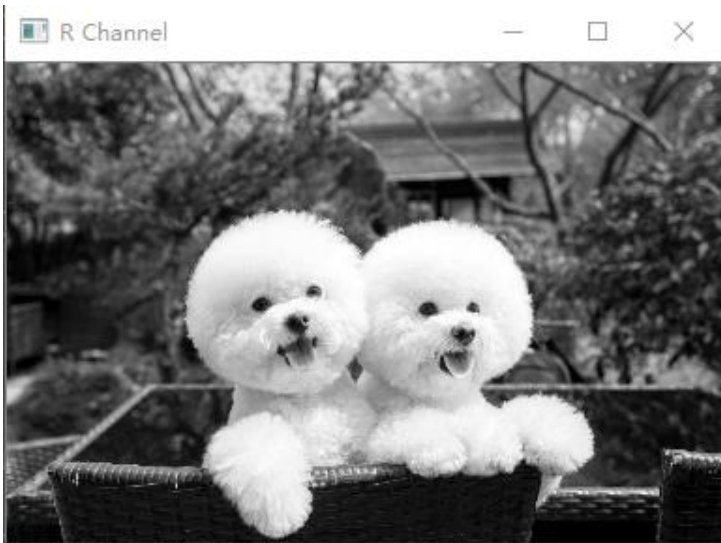
- Use `cv2.imread` to read the image according to its relative paths “images/image1.jpg”
- Use `cv2.resize(image_name, (xsize, ysize))` the first parameter is for the name of the original image which will be resized by cv2. The second parameter is for the output size of the target image. `xsize` is for the length of the image and `ysize` is for the height of the image.

For this question, `xsize` should be 384 and `ysize` should be 256 so that we could have an image of 384 * 256 in columns * rows form. We stored the resulting image in the variable ‘im1’

b.

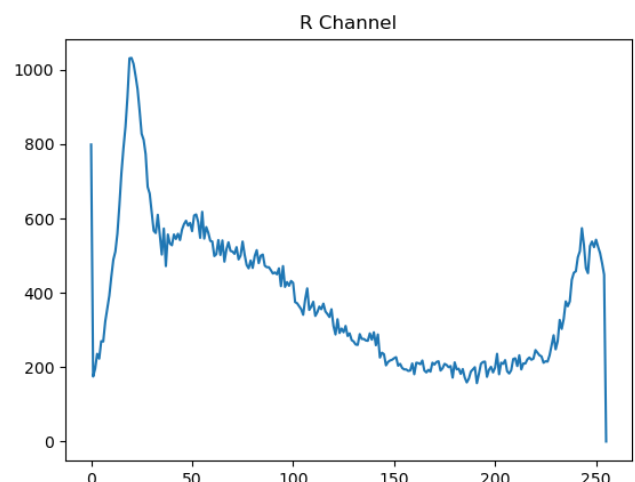
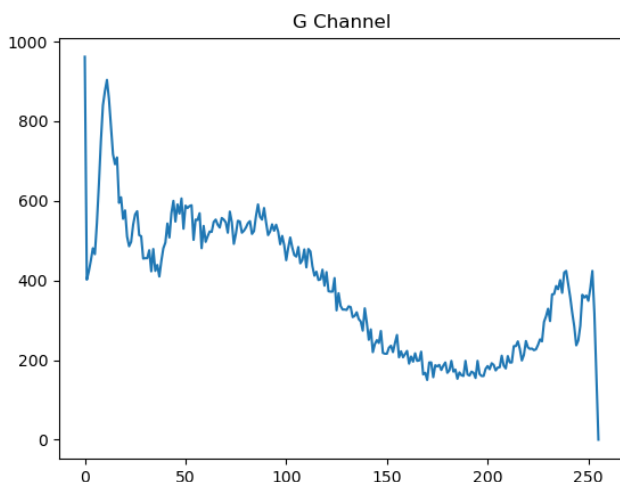
- Firstly, we need to notice that if cv2 is used for reading images, the channels of the image are different from the normal “RGB” order. Instead, the images are stored in “BGR” order. Therefore, if we use the `cv2.split`, it returns three outputs, and the first output is the “B” channel, the second channel is the “G” channel and the third channel is the “R” channel. We stored these channels to variables ‘b’, ‘g’, ‘r’ according to their meanings(E.g.: ‘b’ stored “B” channel)
- Secondly, we still use the `cv2.imshow` to display the three grayscale images separately.

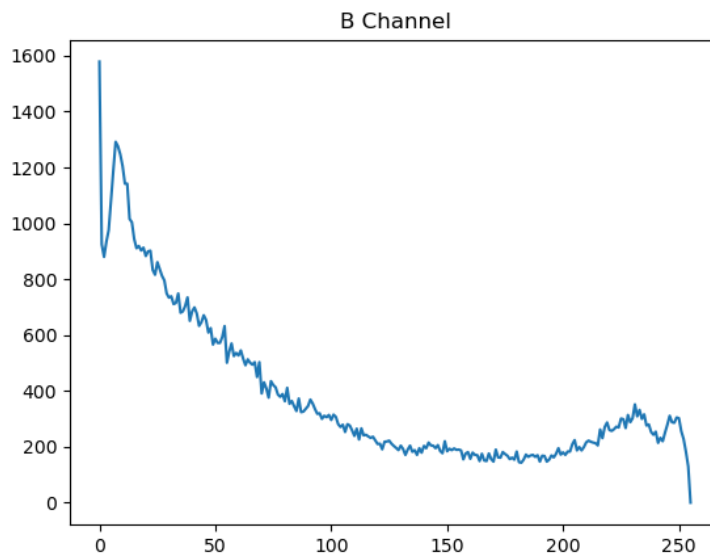




c.

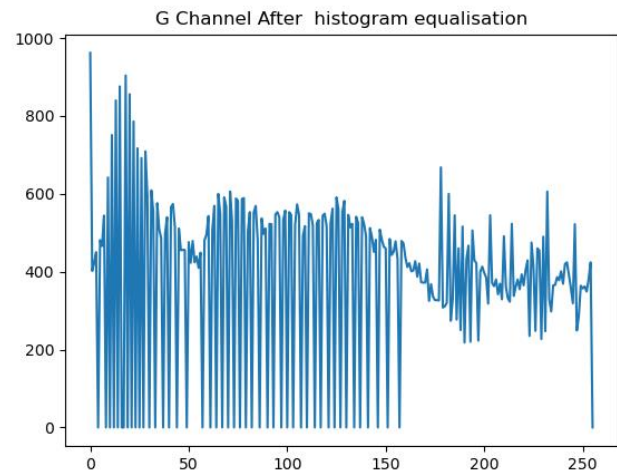
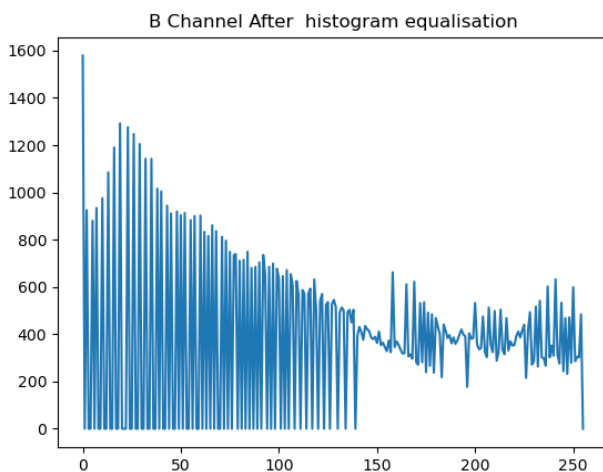
- Convert the grayscale images into histograms. We use the function `cv2.calcHist`. The function has five parameters: The first parameter is for the input image('im1'), the second parameter is for the channel which we want to convert(0 mean B channel, 1 means G channel, 2 means R channel. We could regard it as the array index). The third parameter is for the mask(we do not need to mask some part of the image, so we use "None"). The fourth parameter is about the number of bins, we choose to use 256 because the grayscale varies from 0~255. The last parameter is about the range of the histogram, we choose to use [0,255] as the lowest value of grayscale is 0 and the highest value is 255.
- Display the histograms.
We use 'hsb', 'hsg' and 'hsr' to store the histograms in advance, and now we just need to use matplotlib to display the plots.
We use `plt.title` to add titles for plots and use `plt.plot` to plot these histograms. Finally, use `plt.show` to make the plots visible on the screen.



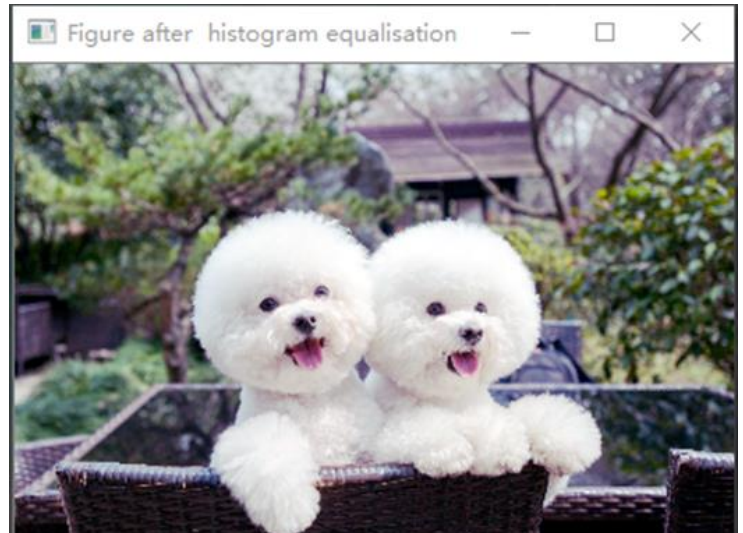
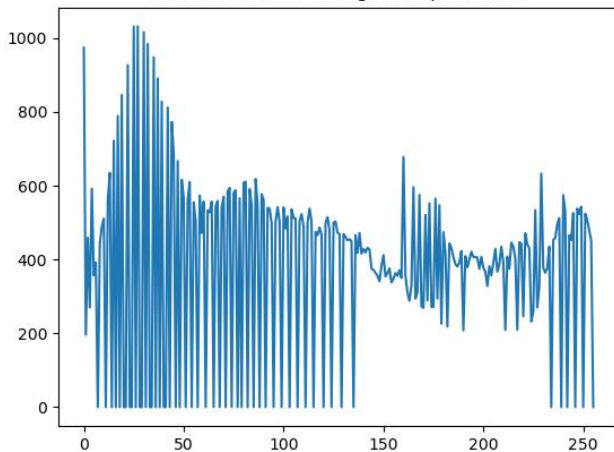


d.

- Calculate the histogram equalization base on images
Use `cv2.equalizeHist` This built-in function takes a grayscale image as an input and output a new image with histogram equalization. Therefore, we could use this function to process histogram equalization on variables "b", "g", "r". We use 'hsb_ec', 'hsg_ec', 'hsr_ec' to store the images after doing histogram equalization.
- Convert images to histograms
Similarly, we use the function `cv2.calcHist` to calculate corresponding grayscale histograms. As the 'hsb_ec', 'hsg_ec' and 'hsr_ec' are grayscale images, they only consist of one channel, so for the second parameter of the function, it will remain 0.
- Display the histograms
Similarly, we use `plt.plot`, `plt.title`, `plt.show` to display the histograms after doing equalization.



R Channel After histogram equalisation



Task-3: Color space conversion (5 marks)

Use the two images in Figure 2 to study color space conversion from RGB to HSV (you can download them from the Wattle site):

1. Based on the formulation of RGB-to-HSV conversion, write your function `cvRGB2HSV()` that converts the RGB image to HSV color space (1.4 marks). Read in Fig. 2(a) and convert it with your function, and then display the H, S, V channels in your report (0.2 marks for each channel, 0.6marks in total).
2. Compute the average H values of five color regions in Fig. 2(b) with your function and the MATLAB's inbuilt function `rgb2hsv()` or Python's inbuilt function `rgb2hsv()`. Print both under the corresponding regions (0.2 marks for each value, 1 mark in total). You also need to explain how to distinguish and divide the five regions, and how to calculate the average Hue value (2 marks, higher marks only for the smarter solution).
(Note: The elements of both colormaps are in the range 0 to 1.)

Q1: Formulas for conversion

- Normalize the RGB values (divided by 255)
- Find X_{max} , $X_{max} = \max(R, G, B) = V$
- Find X_{min} , $X_{min} = \min(R, G, B)$
- Find C , $C = X_{max} - X_{min}$

- Find H,

$$H := \begin{cases} 0, & \text{if } C = 0 \\ 60^\circ \cdot \left(0 + \frac{G-B}{C}\right), & \text{if } V = R \\ 60^\circ \cdot \left(2 + \frac{B-R}{C}\right), & \text{if } V = G \\ 60^\circ \cdot \left(4 + \frac{R-G}{C}\right), & \text{if } V = B \end{cases}$$

- Find S,

$$S_V := \begin{cases} 0, & \text{if } V = 0 \\ \frac{C}{V}, & \text{otherwise} \end{cases}$$

- The main logic for solving this question is to first create a function that could change one specific pixel from “RGB” form to “HSV” form, I created *cvRGB2HSV_pixel* to handle it. I also created the function *cvRGB2HSV* for converting the whole image into “HSV” form.

```
def cvRGB2HSV_pixel(b, g, r):
    normalized_b = b / 255.0
    normalized_g = g / 255.0
    normalized_r = r / 255.0
    # print(normalized_b)
    x_max = max(normalized_b, max(normalized_g, normalized_r))
    v = x_max
    x_min = min(normalized_b, min(normalized_g, normalized_r))
    c = x_max - x_min
    if c == 0:
        h = 0
    elif v == normalized_r:
        if normalized_g >= normalized_b:
            h = ((normalized_g - normalized_b) / c) * 60
        else:
            h = ((normalized_g - normalized_b) / c) * 60 + 360
    elif v == normalized_g:
        h = ((normalized_b - normalized_r) / c + 2) * 60
    elif v == normalized_b:
        h = ((normalized_r - normalized_g) / c + 4) * 60

    if v == 0:
        s = 0
    else:
        s = c / v

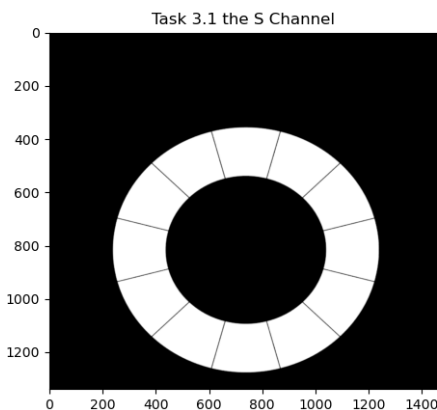
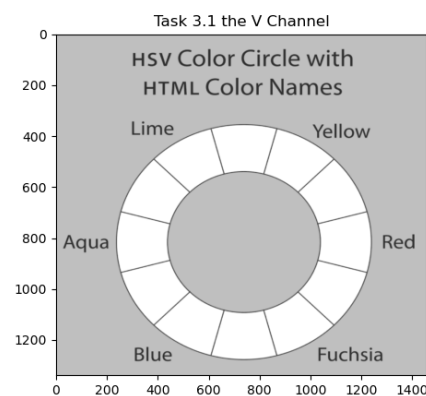
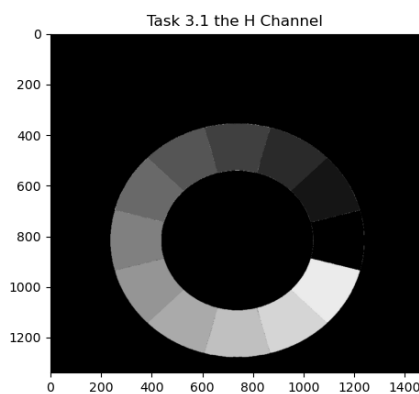
    H = (h / 2.0) / 180.0
    S = s
    V = v
    return (H, S, V)
```

- For function *cvRGB2HSV_pixel*, it takes the values of three channels, and the body of the function just calculate HSV according to the formula(mentioned above).
- However, we find that H could be smaller than 0 when $V = R$ and $(G-B) < 0$. Therefore, we add an extra 360 degrees to convert the negative values into positive.(This idea was mentioned during the lecture)
- After having the result of the three channels we normalize them according to the specification of the question.
 - 1) H varies between 0 and 180, so we divided the value of H by 180.0
 - 2) As we had normalized the inputs at the beginning, $S, V \in [0,1]$. We do not need to normalize them again.

- For function `cvRGB2HSV`, it takes an image with an image of $Image \in (c * h * w)$, where $c = 3$. The body of this function just iterate all $h * w$ pixels and apply `cvRGB2HSV_pixel` to each of them. Therefore, a new “HSV” image is generated.

```
def cvRGB2HSV(image):
    output = np.zeros(image.shape)
    for row in range(0, image.shape[0]):
        for col in range(0, image.shape[1]):
            output[row][col] = cvRGB2HSV_pixel(image[row][col][0], image[row][col][1], image[row][col][2])
    return output
```

Here are the results of Task 3.1:



Q2:

How to distinguish five regions:

- The figure is split equally into five parts, so the length of every region is $2083/5 = 416.6$. It shows that the total length of the figure could not be exactly divided by 5. Therefore, we regard the length of the regions to be 417, and the last region will be 2 pixels shorter than the others. The error could be ignored as the length of the region is much bigger than 2.

How to calculate the average:

- 1) With my function

- Use the function mentioned in Task3.1 to convert the figure into an “HSV” form.
- Use `cv2.split` to get the individual “H” channel.
- We could use the index operation `array[:, start : end]`, *array has n rows and m columns* to extract different regions from the original figure. This operation will return a slice of the input array which has m rows and (*end – start*) columns.
- After that, we could use `np.average` to calculate the average “H” of every region.

2) With built-in function

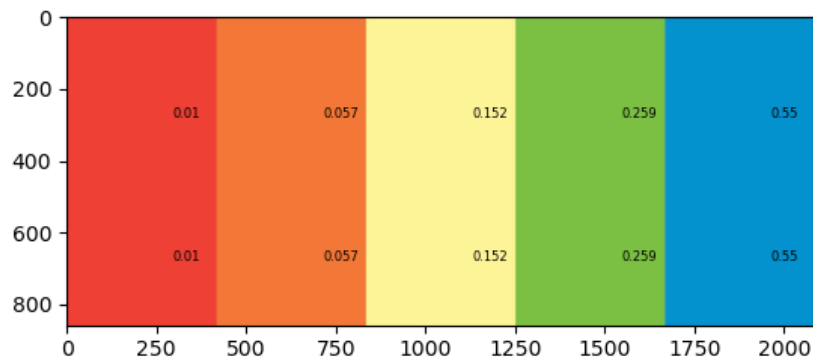
- Import the function from the package “skimage.color”
- Use `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)` to convert the figure into “RGB” form.
- Normalize the “RGB” values (divided by 255)
- Use `rgb2hsv` to get the “HSV” channels
- Use `cv2.split` to get the “H” channel
- We could use the index operation `array[:, start : end]`, *array has n rows and m columns* to extract different regions from the original figure. This operation will return a slice of the input array which has m rows and (*end – start*) columns.
- After that, we could use `np.average` to calculate the average “H” of every region.

How to display the values on the image:

- `plt.text(x, y, s, fontsize = i)`, x refers to the position of the text on the x-axis, y refers to the position of the text on the y-axis, s refers to the content of the text, and font size is set to 6 for better visualization.
- Where to display the text
For the x-axis, every text will be displayed 120 pixels left from the right edge of the corresponding region.
For the y-axis, the output of my function will be placed at 280 pixels below the top edge, and the output of the built-in function will be placed at 680 pixels below the top edge

Justification for the output:

- The outputs are round to three decimals for better visualization, and we can find that the two functions get the same results in every region.
- For more precise comparison, I use `check_for_eq = abs(imb_H – imb_H_in) < 0.000001` and `any_false = np.sum(check_for_eq == 0)` to check if there exist any unacceptable differences between two “H” channels generated by my own function and the built-in function. We find “any_false” equals to “0” by debugging through the program. Therefore, my own function reach to a precise result.



Task-4: Image Denoising via a Gaussian Filter (5 marks)

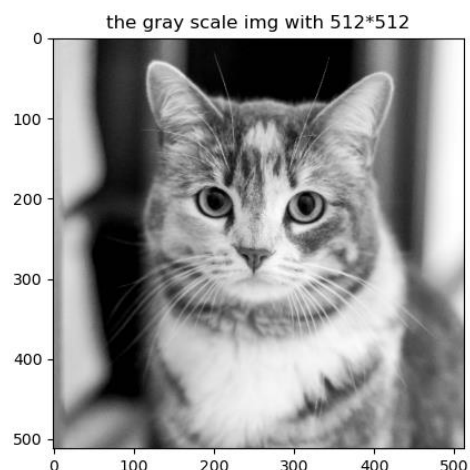
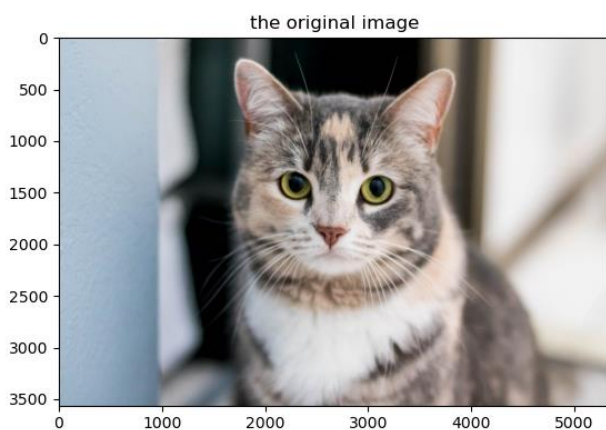
In this task, you are asked to:

1. Read in image2.jpg. Crop a square image region corresponding to the central facial part of the image, resize it to 512*512 and save this square region to a new grayscale image. Please display the two images. Make sure the pixel value range of this new image is within [0, 255] (0.5 marks).
2. Add Gaussian noise to this new 512x512 image (Review how you generate random number in Task-1). Use Gaussian noise with zero mean, and standard deviation of 15 (0.5 marks). The intensity values of the Generated image should be within[0, 255].
Hint: Make sure your input image range is within [0, 255]. Kindly, you may need np.random.randn() in Python. While Matlab provides a convenient function imnoise(). Please check the default setting of these inbuilt functions.
3. Display the two histograms side by side, one before adding the noise and one after adding the noise (0.25 marks for each histogram, 0.5 marks in total).
4. Implement your own Matlab function that performs a 5x5 Gaussian filtering (1.5 marks). Your function interface is:
my_Gauss_filter()
input: noisy_image, my 7x7 gausskernel
output: output_image

5. Apply your Gaussian filter to the above noisy image and display the smoothed images and visually check their noise-removal effects (0.5 marks in total). One of the key parameters to choose for the task of image filtering is the standard deviation of your Gaussian filter. You may need to test and compare different Gaussian kernels with different standard deviations (1.0 marks). Note: In doing this task you MUST NOT use any Matlab's (or Python's) inbuilt image filtering functions (e.g., `imfilter()`, `filter2()` in Matlab, or `cv2.filter2D()` in Python). In other words, you are required to code your own 2D filtering code, based on the original mathematical definition for 2D convolution. However, you are allowed to generate a 7x7 sized Gaussian kernel with inbuilt functions.
6. Compare your result with that by Matlab's inbuilt 7x7 Gaussian filter (e.g. `filter2()`, `imfilter()` in Matlab, or conveniently `cv2.GaussianBlur()` in Python,). Please show that the two results are nearly identical (0.5 marks)

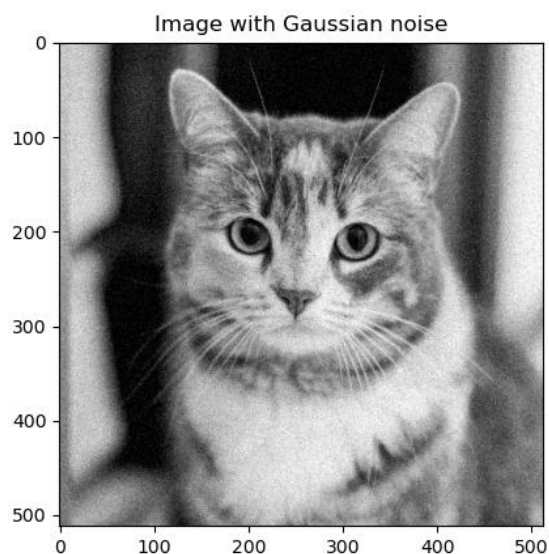
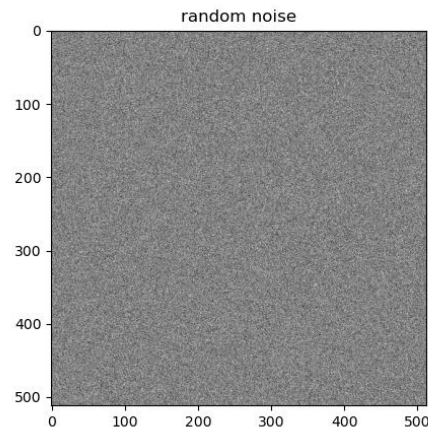
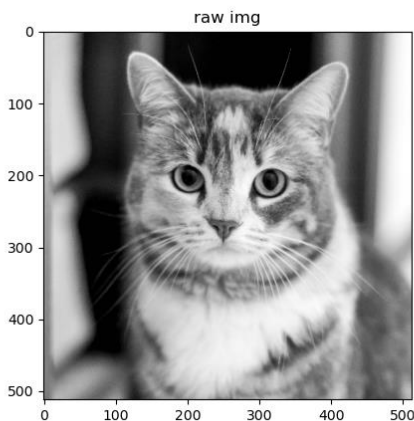
Answers:

1. For convenience and performance, I first resize image4 with the same standard as image1(resize the figure into 1042*720. After that, because the final output should be a picture of size 512*512, so I crop the image according to the length of its shorter edge. Therefore, I get a square image before resizing it.
 - `cv2.resize()` is used for resizing process, the first parameter is the target image, and the second parameter indicates the length and width after resizing (for this question it should be (512,512)).
 - I use `cv2.cvtColor()` to change the image from "GBR" to grayscale. The second parameter show be `cv2.COLOR_BGR2GRAY` because in cv2 the channel order is "BGR" instead of "RGB"
 - `cv2.imwrite()` helps to save the resulting image. Its first parameter indicates the directory for storing and the second parameter indicates which image object will be stored. For this question, I store the grayscale image with size 512*512 on the directory *images/image4_gray.jpg*.
 - Here are the "image4_gray.jpg" and the "image4.jpg"(the original image), you could also check the images stored in the directory to find out its resolution.



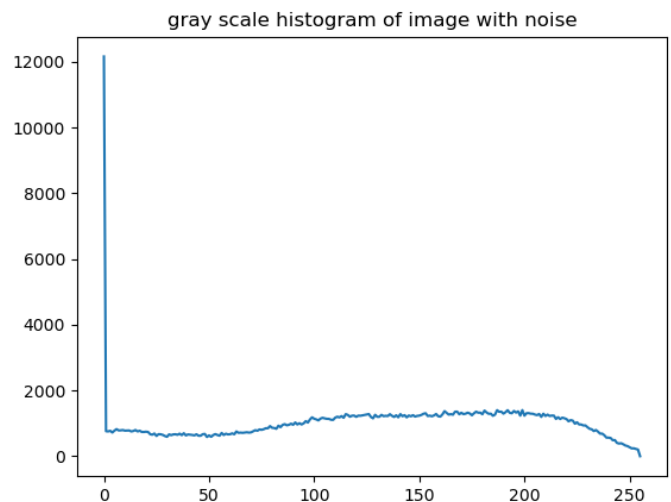
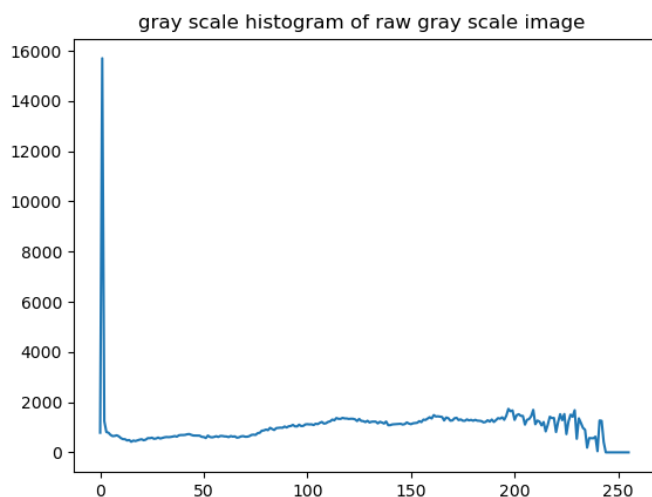
2.

- First, `np.random.normal()` is used to generate gaussian noise. For this question, it should be `np.random.normal(0,15,gray_img.size)`. Zero stands for the mean of the gaussian distribution, fifteen stands for the deviation of the distribution, “gray_img.size” stands for the size of the output because we want to generate a noise array with the same size of the grayscale image.
- Second, I just add the noise array with the grayscale image.
- Third, I use `np.clip(raw_noise_img, 0.0, 255.0)`, where `raw_noise_img` stands for the grayscale image with noise, make sure that all pixel values are between 0 and 255. The second parameter indicates the lower bound of a pixel value and the third parameter indicates the upper bound of a pixel value.
- Finally, `np.uint8()` helps to convert the data type of the image into `uint8` (`uint8 ∈ [0,255]`). The original image and image under noise are shown below, and the effect of noise is obvious.



3.

- `cv2.calcHist([noise_img], [0], None, [256], [0, 255])` and `cv2.calcHist([gray_img], [0], None, [256], [0, 255])` are used to generate histograms of two images. For the function `calcHist()`, it has five parameters: The first parameter is for the input image (“noise_img” or “gray_img”), the second parameter is for the channel which we want to convert (As they are grayscale images, they only consist of one channel, so for the second parameter of the function it will remain 0). The third parameter is for the mask (we do not need to mask some part of the image, so we use “None”). The fourth parameter is about the number of bins, we choose to use 256 because the grayscale varies from 0~255. The last parameter is about the range of the histogram, we choose to use [0,255] as the lowest value of grayscale is 0 and the highest value is 255.



4.

- Generate Gaussian kernel

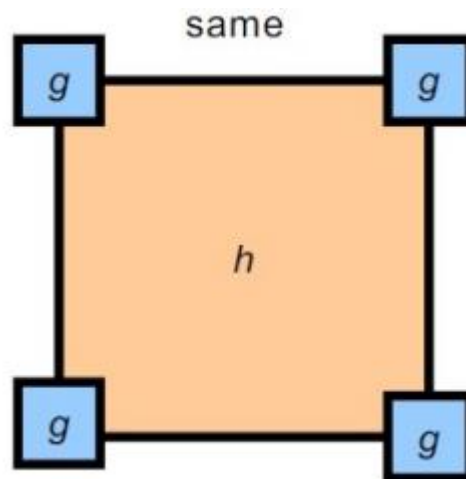
```
def get2dGaussianKernel(sigma, kernel_size=7, ktype=cv2.CV_32F):
    kernel_1d = cv2.getGaussianKernel(kernel_size, sigma, ktype)
    return kernel_1d * kernel_1d.T
```

- 1) `cv2.getGaussianKernel()` is a function that generates 1d Gaussian kernel. The first parameter is the Gaussian distribution's sigma. The second parameter defines the size of the kernel (length of the output 1d array). For the third parameter, we just keep the data type to be CV_32F which is the data type used most frequently.
- 2) After generating a 1d kernel we could use `A * A.T` (A transpose) to get a 2d gaussian kernel. According to the separability of the filter, the correctness of my function could be proved.

- Add padding to the target image


```
def addPadding(image, kernel_size):
    padding = kernel_size // 2
    padding_img = np.zeros((image.shape[0] + padding * 2, image.shape[1] + padding * 2), dtype="float64")
    padding_img[padding:padding + image.shape[0], padding:padding+image.shape[1]] = image
    return padding_img
```

- 1) From the lecture slides we can find the relationship between the kernel size and image padding. $\text{Padding} = 0.5 * \text{kernel size}$



- 2) We create a 2d array filled with zeros and copy the original image(the image array) to the central part of the array.

- Implement Gaussian Filter

```
def my_Gauss_filter(noise_image, gaussian_kernel):
    padding_img = addPadding(noise_image, gaussian_kernel.shape[0])
    padding = gaussian_kernel.shape[0] // 2
    filtered_img = np.zeros(padding_img.shape, dtype="float64")
    for row in range(0, noise_image.shape[0]):
        for col in range(0, noise_image.shape[1]):
            kernel_template = padding_img[row:row+gaussian_kernel.shape[0], col:col+gaussian_kernel.shape[0]]
            #print(kernel_template)
            sum_result = np.sum(kernel_template * gaussian_kernel)
            filtered_img[row+padding][col+padding] = sum_result
    filtered_img = filtered_img[padding:padding+noise_image.shape[0], padding:padding+noise_image.shape[1]]
    filtered_img = np.clip(filtered_img, 0, 255)
    return np.uint8(filtered_img)
```

- 1) Parameters

The first parameter is for the noise image, the second parameter is for the filter kernel

- 2) sliding window

$f[m, n] = \sum_{k, l} K[k, l] h[m + k, n + l]$, where K stands for filter kernel, h stands for the original image, f stands for output image, k stands for index of filter kernel on the x-axis, and l stands for the index of filter kernel on the y-axis.

This algorithm is used to generate new images according to what has been mentioned in the lecture slides' example.

Instead of using built-in functions to implement the sliding window algorithm, I use nested loops which could iterate through all possible

windows of the image.

3) Crop the image

We need to crop the image after doing a Gaussian filter because we add padding at the beginning of the filtering process.

4) Data type conversion

I use `np.clip(filter_img, 0, 255)`,

where *filter_img* stands for the output image

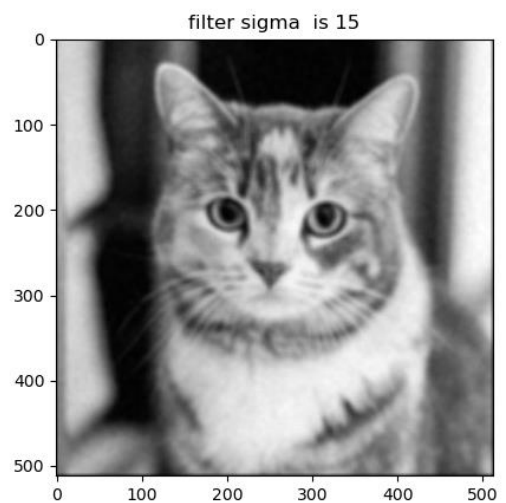
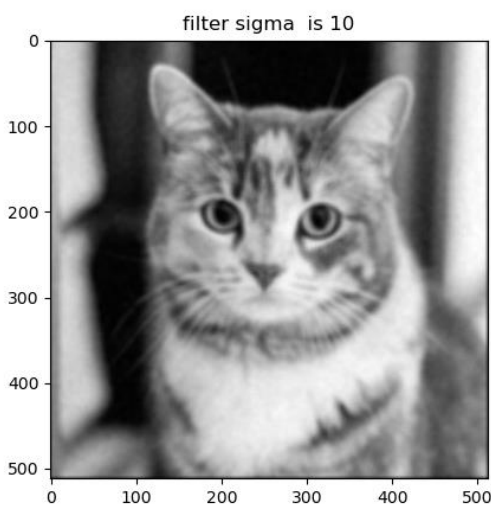
to make sure that all pixel values are between 0 and 255. The second parameter indicates the lower bound of a pixel value and the third parameter indicates the upper bound of a pixel value. Finally, `np.uint8()` helps to convert the data type of the image into `unit8` ($unit8 \in [0, 255]$).

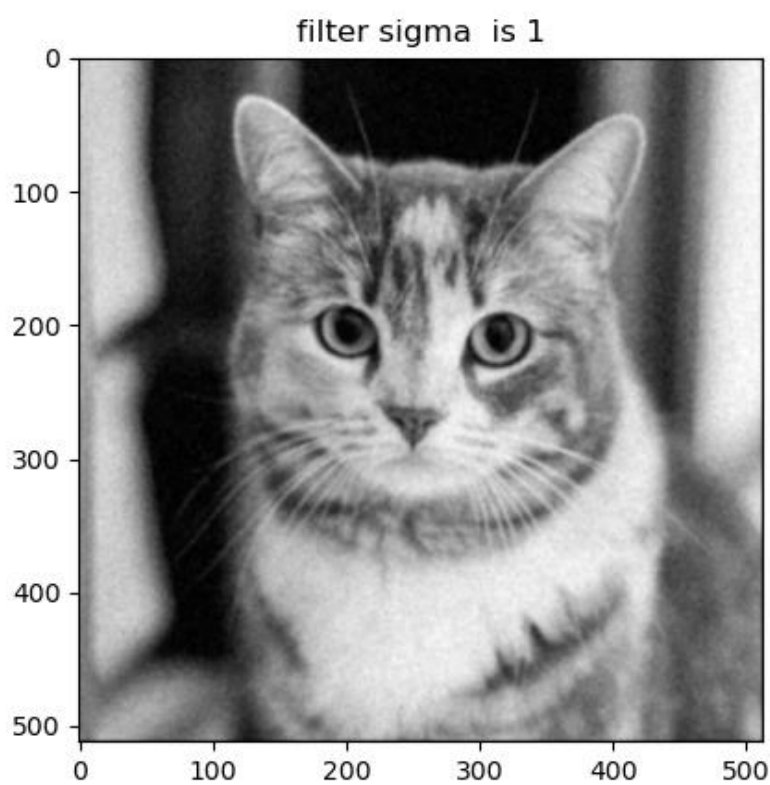
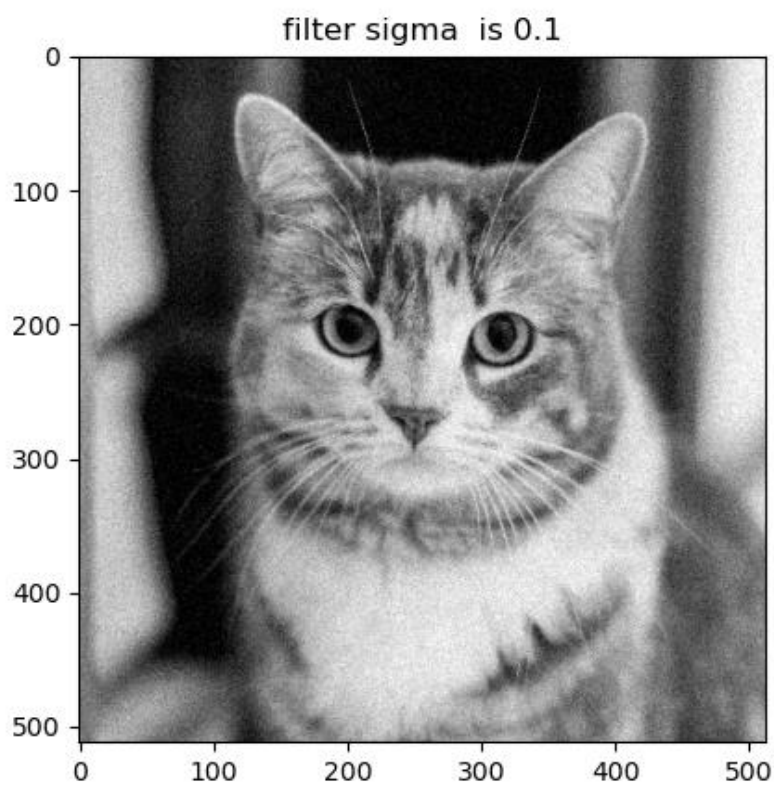
5.

- How to apply the filter

- 1) Use `get2dGaussianKernel` to generate kernels for filtering. I just keep the default parameters of this function unchanged for this question.
- 2) Use `my_Gauss_filter` to do the filtering. Pass the target image and related kernel to the function and it will automatically add padding before starting the filtering process

- Output image with different Gaussian kernels





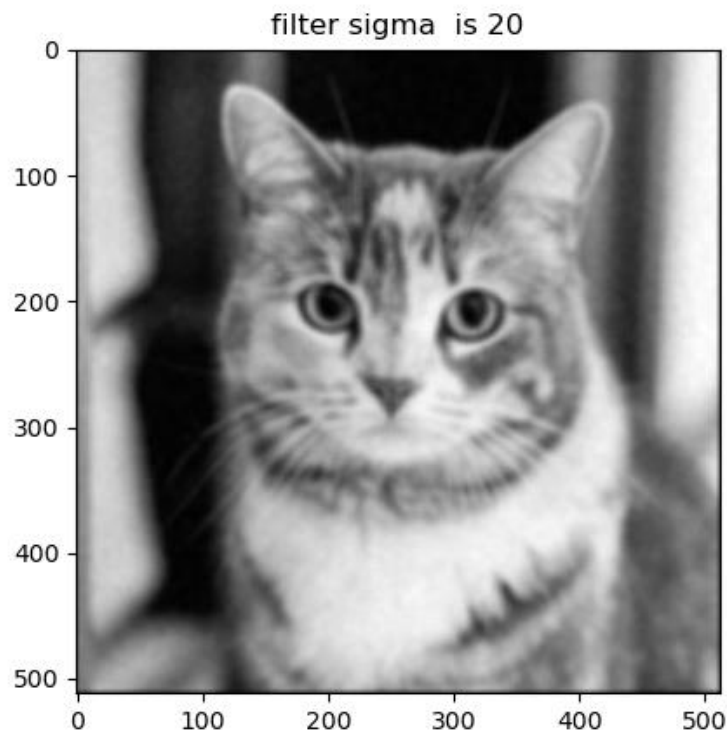
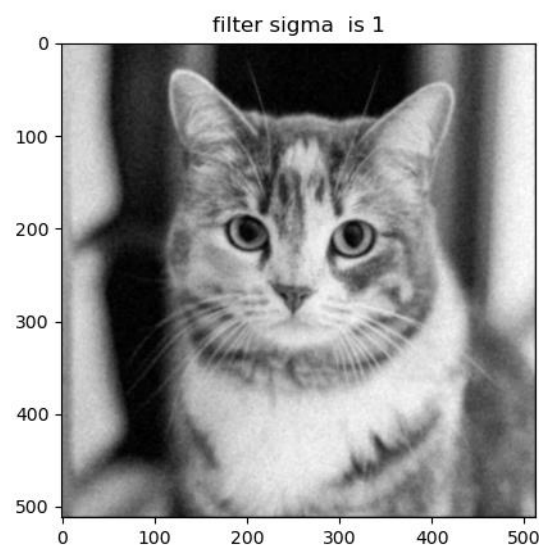
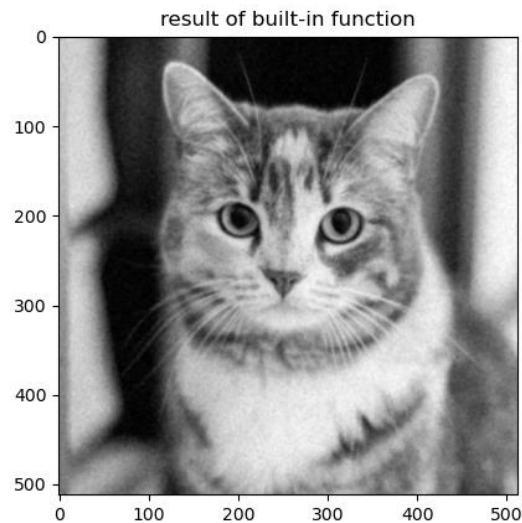


Image is blurred when the sigma value becomes larger, so we want sigma to be as small as possible. However, when sigma is smaller than 1 there is much more noise in the output image. Therefore, in my opinion, 1 would be an optimal parameter of sigma because the output is clear enough with lower noise.

6.

- `cv2.GaussianBlur(noise_img, (7,7),1)` is used to do the gaussian filtering. The second parameter indicates the kernel size is 7*7 and the third parameter indicates the sigma is 1.





- We can find that these two images are nearly identical by comparing them with our eyes.

Task-5: Implement your own 3x3 Sobel filter in MATLAB/Python

1. You need to implement your own 3x3 Sobel filter. Again, you must not use any inbuilt edge detection filter. The implementation of the filter should be clearly presented with your code. Test it on sample images (image2.jpg, image4.jpg provided in the Lab 1 directory on wattle) (we suggest converting it to grayscale) (1.5 marks) and compare your result with the inbuilt Sobel edge detection function (0.5 marks). Briefly explain the function of the Sobel filter and how it can achieve this function (1.0 mark).
2. Investigate the accuracy of the Sobel filter in terms of predicting the orientation of the edge. If it is used for edge detection, what might cause any inaccuracy of the edge orientation estimation that you see? Look at the histogram of the gradient orientation and find out the major edge orientations in the image. Please discuss whether the gradient orientation histogram aligns with the real expected edges for the image or not. You can use the inbuilt Sobel to discuss this question. (2.0 mark)

Answer:

1.

- Implement my own Sobel filter
 - 1) Reusability
The function *addPadding()* could be reused for this task to generate an image with padding.

2) Implement *my_Sobel_filter*

- Parameters

image: target image

sobel_x: The Sobel kernel on the x-axis, which detects the vertical edges

sobel_y: The Sobel kernel on the y-axis, which detects the horizontal edges.

For this question $\text{sobel_x} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$, $\text{sobel_y} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$

- Logic

The logic of the Sobel filter is similar to any other filter operation. Nested loops are used for implementing the sliding window algorithm.

On the x-axis, every slice will do an element-wise multiplication with “sobel_x” and sum every result values together (G_x). For the y-axis, we calculate G_y in a similar way.

After that the edge strength is given by the gradient magnitude

$$\sqrt{(G_x^2 + G_y^2)}.$$

- Crop the image

We need to crop the image after being filtered because we add padding at the beginning of the filtering process.

- Data type conversion

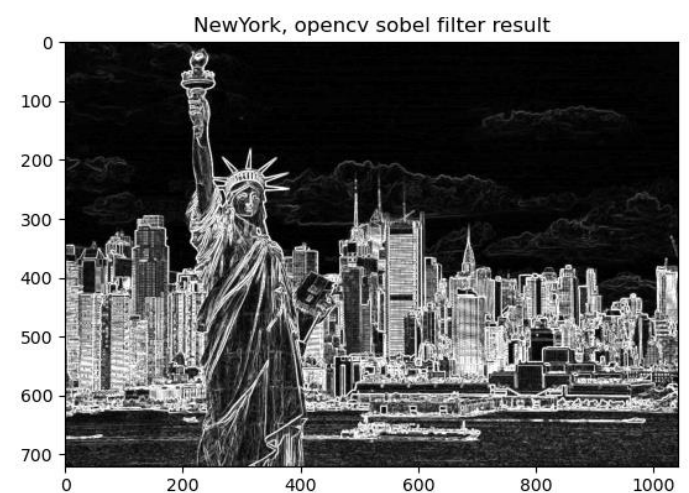
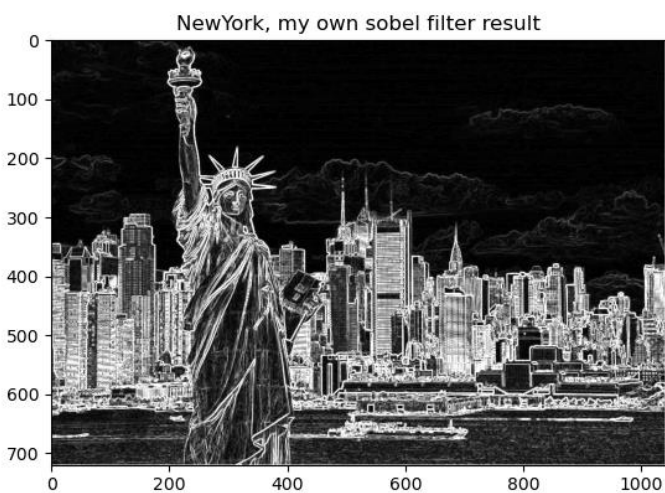
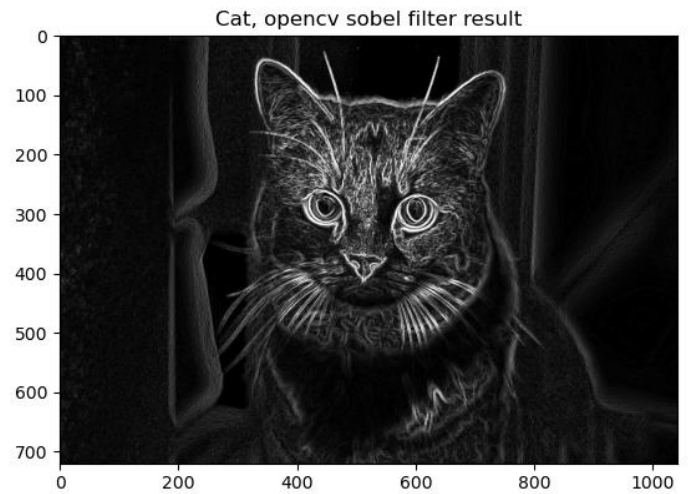
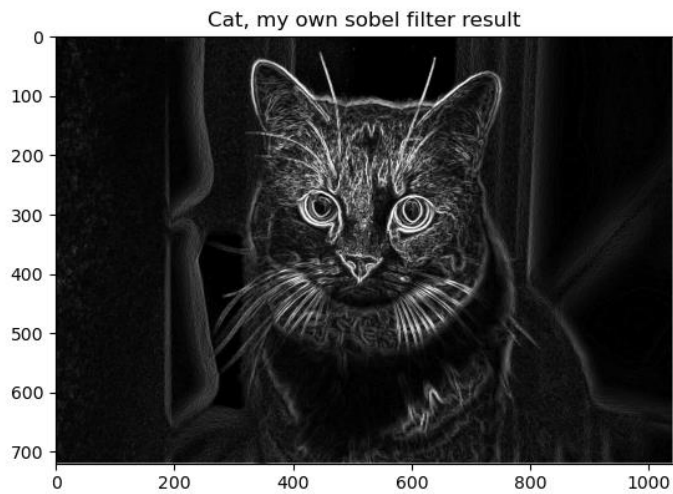
I use `np.clip(filter_img, 0, 255)`,

where *filter_img* stands for the output image

to make sure that all pixel values are between 0 and 255. The second parameter indicates the lower bound of a pixel value and the third parameter indicates the upper bound of a pixel value. Finally, `np.uint8()` helps to convert the data type of the image into `unit8` (`unit8 ∈ [0, 255]`).

```
def my_Sobel_filter(image, sobel_x = sobel_x, sobel_y = sobel_y):
    padding_img = addPadding(image, sobel_x.shape[0])
    padding = sobel_x.shape[0] // 2
    filtered_img = np.zeros(padding_img.shape, dtype="float64")
    for row in range(0, image.shape[0]):
        for col in range(0, image.shape[1]):
            kernel_template = padding_img[row:row+sobel_x.shape[0], col:col+sobel_x.shape[0]]
            #print(kernel_template)
            sum_result = math.sqrt(np.sum(kernel_template * sobel_x)**2 + np.sum(kernel_template * sobel_y)**2)
            filtered_img[row+padding][col+padding] = sum_result
    filtered_img = filtered_img[padding:padding+image.shape[0], padding:padding+image.shape[1]]
    filtered_img = np.clip(filtered_img, 0, 1)
    return (filtered_img)
```

- Display the outputs



From these four images, we can get that my implementation of the Sobel filter could give out the same output as the OpenCV built-in Sobel filter function because I could not find any differences between these images.

- Explain the mechanism of the Sobel filter

Sobel filter uses the first derivative to detect the edges. The filter will regard the place where the grayscale value changes rapidly. From the first derivative perspective, local minimum or maximum will appear in the corresponding location.

The image array could be regarded as an array of discrete data, so we could only approximate the derivative by calculating $f(x+1, y) - f(x, y)$ and $f(x, y+1) - f(x, y)$. If we use matrix operation to represent the derivative,

$f(x+1, y) - f(x, y) \Rightarrow [-1 \ 1], f(x, y+1) - f(x, y) \Rightarrow \begin{bmatrix} -1 \\ 1 \end{bmatrix}$. However,

the image array is discrete, the result value's location will lie between two consecutive pixels which could not be represented by an array. Therefore, the kernel size should be an odd value. On the other hands, we could use $f(x+1, y) - f(x-1, y)$ to approximate the partial derivative of x, and its corresponding matrix operation could be defined as $\begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$. For a 2d

image we could use the matrix $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ to calculate the partial derivative

of x at the central point $f(x, y)$ (because we want to have a smooth output, so we add extra gaussian filter when doing edge detecting and that's why we have $\begin{bmatrix} -2 & 0 & 2 \end{bmatrix}$ for the central of the window.). Similarly, we could use

$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$ to find partial derivative of y-axis. Finally, the edge strength

is given by $\sqrt{(G_x^2 + G_y^2)}$

where G_x is the partial derivative of x

- axis, G_y is the partial derivative of y - axis

2.

- My discovery and opinions about inaccuracy.

From the cat picture, I find that the Sobel seems to detect cat fur as edges => the Sobel edge filter will be misled when facing different textures.

From the New York image, I find that a large number of edges of the buildings are ignored in the output => the Sobel filter will become inaccurate when the original image is complex. Many edges could not be represented in the output image.

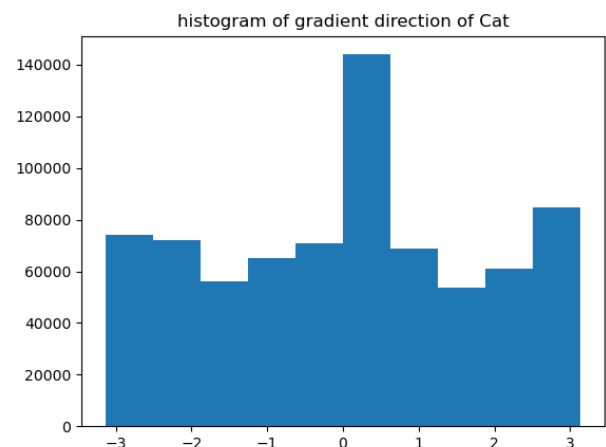
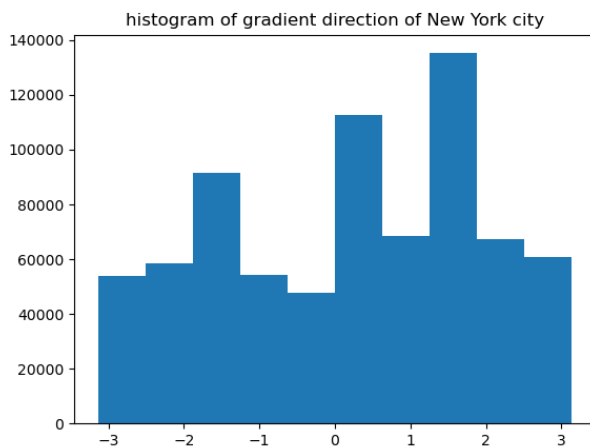
In theory, the Sobel filter uses the first derivative to detect edges, which means that the filter will be influenced by the noise. The reliability of Sobel is lower than Laplace when facing noise images.

- How to find the histogram of gradient direction

Use `math.atan2(y, x)` to calculate the arctangent of the gradient. `math.atan2()` $\in [-\pi, \pi]$.

Use `plt.hist()` to plot the histograms.

```
def my_Sobel_hist(image_xx, image_yy):
    result = []
    for row in range(0, img_xx.shape[0]):
        for col in range(0, img_xx.shape[1]):
            result.append(math.atan2(img_yy[row][col], img_xx[row][col]))
    return result
```



3. Major edge orientations in the image

The cat image: horizontally. However, the edges of the cat should not have so many horizontal edges. There should not be a specific majority orientation of this image because the cat has a round face. The histogram does not align with the real expected edges for the image.

The New York image: horizontally and vertically. There are many buildings in the image so there is a large number of horizontal edges and vertical edges. histogram aligns with the real expected edges for the image.